

Projekt z przedmiotu Obsługa Dużych Zbiorów Danych

Tytuł projektu: Zaprojektowanie chmurowego systemu do analizy wydajności bazy MongoDB z wykorzystaniem narzędzia JMeter.

Udowodnienie hipotezy o istotności różnic miar wydajności.

- [1. Cel projektu](#)
- [2. Opis wymagań systemu informacyjnego \(zakres i ograniczenia\)](#)
 - [Zakres funkcjonalny](#)
 - [Ograniczenia](#)
- [3. Opis stosu technologicznego](#)
 - [System zarządzania bazą danych](#)
 - [Zbiory danych](#)
 - [Środowisko chmurowe](#)
 - [Opis stosu technologicznego:](#)
- [4. Baza Netflix Movies and TV Shows](#)
 - [4.1 Opis bazy](#)
 - [4.2 Opis stanowiska do analiz](#)
 - [4.3 Metodyka badań wydajności](#)
 - [4.4 Wyniki i wnioski](#)
 - [4.5 Instrukcja uruchamiania systemu](#)
- [5. Baza customers_organizations](#)
 - [5.1 Opis bazy](#)
 - [Kolekcja: customers_10000](#)
 - [Kolekcja: organizations_1000000](#)
 - [5.2 Opis stanowiska do analiz](#)
 - [5.3 Metodyka badań wydajności](#)
 - [Kolekcja: customers_10000](#)
 - [Kolekcja: organizations_1000000](#)
 - [5.4 Wyniki i wnioski](#)
 - [Kolekcja: customers_10000](#)
 - [Kolekcja: organizations_1000000](#)
 - [5.5 Instrukcja uruchamiania systemu](#)
- [6. Spis literatury](#)
- [7. Spis załączników](#)

Baza Netflix TV Shows and Movies

Warstwa techniczna i integracyjna

- konfiguracja połączenia z chmurową bazą danych MongoDB Atlas,
- przygotowanie struktury bazy danych oraz kolekcji dokumentów (titles, credits),
- import danych z plików CSV do MongoDB,
- implementacja zapytań testowych do bazy danych (operacje find()),
- integracja środowiska Python z bazą danych przy użyciu biblioteki PyMongo,

Analiza wydajności i opracowanie wyników

- przygotowanie scenariuszy testów wydajnościowych dla różnych kolekcji danych,
- pomiar czasów odpowiedzi zapytań do bazy danych,
- obliczenie miar statystycznych (średnia, mediana),
- przeprowadzenie testów statystycznych (test t-Student, Welch) w celu weryfikacji hipotezy badawczej,
- wizualizacja wyników analizy (wykresy czasów odpowiedzi),
- opracowanie części raportowej dotyczącej wyników i wniosków.
- wstępna weryfikacja poprawności danych i zapytań.

1. Cel projektu

Celem projektu było zaprojektowanie i implementacja chmurowego systemu analizy wydajności bazy danych MongoDB, umożliwiającego:

- pomiar czasów odpowiedzi dla różnych operacji bazodanowych,
- porównanie wydajności zapytań wykonywanych na różnych kolekcjach dokumentów,
- statystyczną weryfikację hipotezy o istotności różnic miar wydajności.

Projekt miał charakter eksperymentalno-analityczny i został zrealizowany wyłącznie z użyciem publicznych i darmowych usług chmurowych.

2. Opis wymagań systemu informacyjnego (zakres i ograniczenia)

Zakres funkcjonalny

- połączenie z chmurową bazą MongoDB,
- import i przechowywanie danych dokumentowych,
- udostępnienie API do wykonywania zapytań testowych,
- pomiar czasów odpowiedzi,
- analiza statystyczna wyników (średnia, mediana, test t-Studenta, test Welch).

Ograniczenia

- wykorzystanie wyłącznie darmowych planów usług,
- środowisko wykonawcze: Google Colab,
- ograniczenia wydajnościowe narzucone przez darmowy klaster MongoDB Atlas.

3. Opis stosu technologicznego

System zarządzania bazą danych

Zastosowano **MongoDB Atlas**, czyli chmurową wersję nie relacyjnej bazy danych MongoDB, charakteryzującą się:

- modelem dokumentowym (JSON/BSON),
- elastycznym schematem danych,
- wysoką skalowalnością w środowisku chmurowym.

Zbiory danych

W projekcie wykorzystano dwa zbiory danych:

Netflix Movies and TV Shows dostępny na platformie **Kaggle**, obejmujący:

- titles.csv – ok. 6 000 dokumentów,
- credits.csv – ok. 78 000 dokumentów.

Dane zostały pobrane do Google Colab przez kaggle-cli (biblioteka **Kaggle**), poddane inspekcji, oczyszczone (**Pandas**), a na koniec przekształcone do postaci dokumentów i zaimportowane do MongoDB Atlas przy użyciu biblioteki **PyMongo**.

Środowisko chmurowe

Analizy przeprowadzono w **Google Colab**, który zapewnia:

- dostęp do środowiska Python bez instalacji lokalnej,
- integrację z **Jupyter Notebook**,
- możliwość bezpiecznego przechowywania kluczy dostępowych.

Opis stosu technologicznego:

Warstwa	Co opisuje?	Platforma / środowisko / narzędzia
Sprzęt / środowisko uruchomieniowe	Miejsce uruchamiania kodu i analiz	Google Colab (środowisko chmurowe oparte o infrastrukturę Google Cloud, dostęp przez przeglądarkę, brak konieczności instalacji lokalnej)
System operacyjny	System bazowy środowiska wykonawczego	Linux (Ubuntu – środowisko serwerowe Google Colab), systemy serwerowe MongoDB Atlas (Linux)
Warstwa baz danych	Silnik bazy danych i sposób przechowywania danych	MongoDB Atlas (chmurowa, dokumentowa baza NoSQL), format BSON/JSON
Źródła danych	Dane wykorzystywane w analizach	Publiczne zbiory danych Netflix Movies and TV Shows z platformy Kaggle (titles.csv, credits.csv) Datablist - customers.csv i organizations.csv
Warstwa integracji / API	Komunikacja aplikacji z bazą danych	PyMongo – oficjalny klient MongoDB dla języka Python (połączenie z klastrem Atlas, wykonywanie zapytań find, insert_many)

Warstwa	Co opisuje?	Platforma / środowisko / narzędzia
Warstwa obliczeniowa / analityczna	Przetwarzanie danych i analiza statystyczna	Python 3.12 , biblioteki: numpy (obliczenia numeryczne), pandas (operacje na danych), scipy.stats (test t-Studenta, test Mann–Whitney U, obliczenia statystyczne), statistics (średnia, mediana)
Warstwa wizualizacji / raportowania	Prezentacja wyników analizy	Matplotlib (wykresy czasów odpowiedzi), opcjonalnie Plotly (interaktywne wizualizacje) boxplots i histogramy rozkładów czasów odpowiedzi
Środowisko analityczne	Interfejs do pracy z kodem i wynikami	Jupyter Notebook uruchamiany w Google Colab (komórki kodu + komentarze opisowe)
Warstwa testowa	Testowanie wydajności systemu	Testy syntetyczne realizowane w Pythonie (alternatywa dla JMeter), wielokrotne zapytania do różnych kolekcji
Bezpieczeństwo i dostęp	Autoryzacja i dostęp do zasobów	URI MongoDB Atlas z uwierzytelnianiem użytkownika, whitelist IP w Atlas, zmienne środowiskowe w Colab

4. Baza Netflix Movies and TV Shows

Źródło: <https://www.kaggle.com/datasets/victorsoeiro/netflix-tv-shows-and-movies>

Notatnik: [mongo-atlas-performance-test_netflix-db.ipynb](#)

4.1 Opis bazy

Notatnik zawiera sekcję o nazwie "3. Data Visualisation" w pełni poświęconą eksploracji i wizualizacji zbioru.

```
3. Data Visualisation
3.1 titles collection charts
3.2 credits collection charts
3.3 Join both collections (titles & credits)
```

Przykładowe wizualizacje:

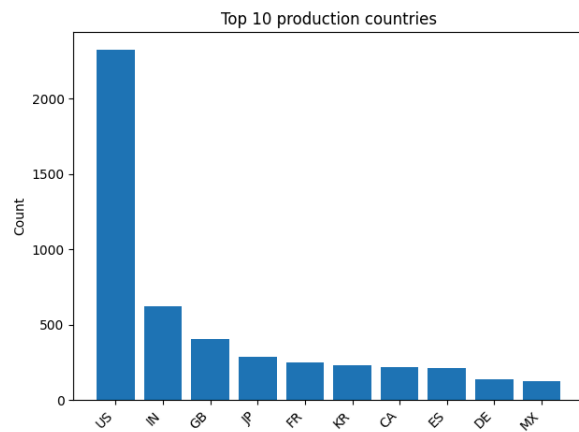
- Top 10 krajów produkcji (`titles`)

```
Top 10 krajów produkcji

docs = list(db.titles.find({}, {"_id": 0, "production_countries": 1}))
countries = []
for d in docs:
    if d.get("production_countries"):
        countries.extend(d["production_countries"])

top = Counter(countries).most_common(10)
labels, values = zip(*top) if top else ([], [])

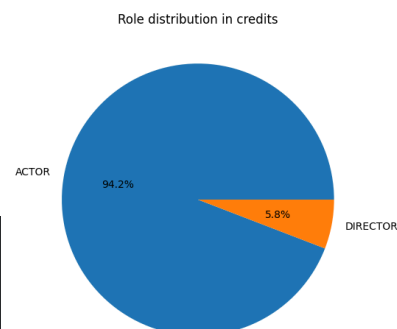
plt.figure()
plt.bar(labels, values)
plt.xticks(rotation=45, ha="right")
plt.ylabel("Count")
plt.title("Top 10 production countries")
plt.tight_layout()
plt.show()
```



- Rozkład ról w zbiorze ludzi odpowiedzialnych za produkcję (`credits`)

```
role_counts = credits_df["role"].value_counts()

plt.figure(figsize=(6, 6))
plt.pie(role_counts, labels=role_counts.index, autopct="%1.1f%%")
plt.title("Role distribution in credits")
plt.show()
```



- Jak zmieniała się "wielkość obsady" w czasie? (``Join both collections (titles & credits)``)

Jak zmieniała się "wielkość obsady" w czasie?

```
from collections import defaultdict

# credits per title
pipeline = [
    {"$group": {"_id": "$id", "credits_count": {"$sum": 1}}}
]

credits_counts = {
    doc["_id"]: doc["credits_count"]
    for doc in credits_col.aggregate(pipeline)
}

# agregacja po dekadach
decades = defaultdict(list)

for t in titles_col.find(
    {},
    {"_id": 0, "id": 1, "release_year": 1}
):
    year = t.get("release_year")
    if year and t["id"] in credits_counts:
        decade = (year // 10) * 10
        decades[decade].append(credits_counts[t["id"]])
```

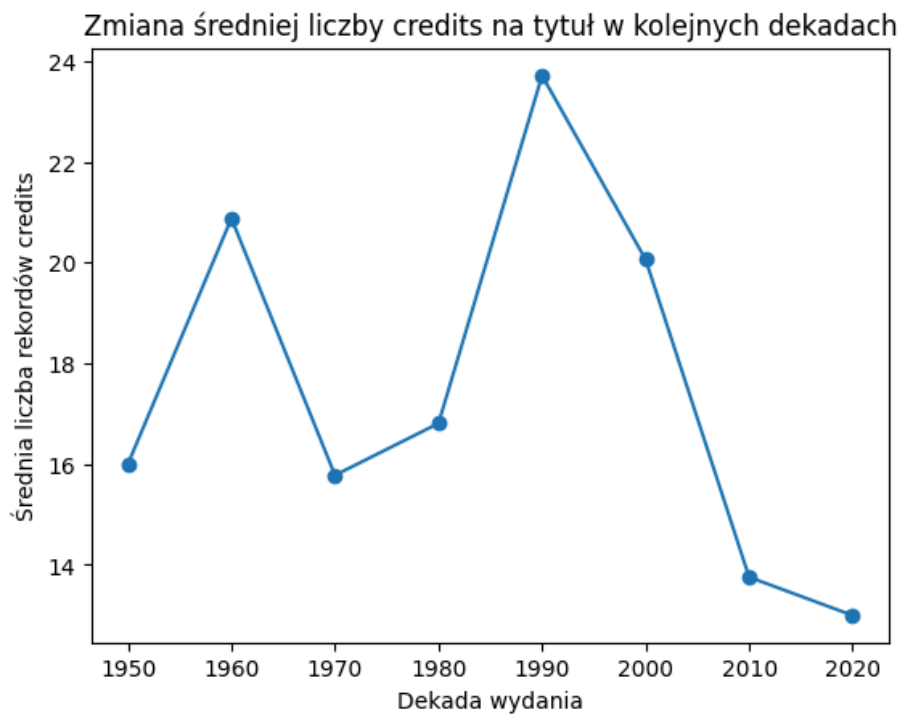
```
import matplotlib.pyplot as plt

decades_sorted = sorted(decades.keys())
avg_cast = [
    sum(decades[d]) / len(decades[d])
    for d in decades_sorted
]

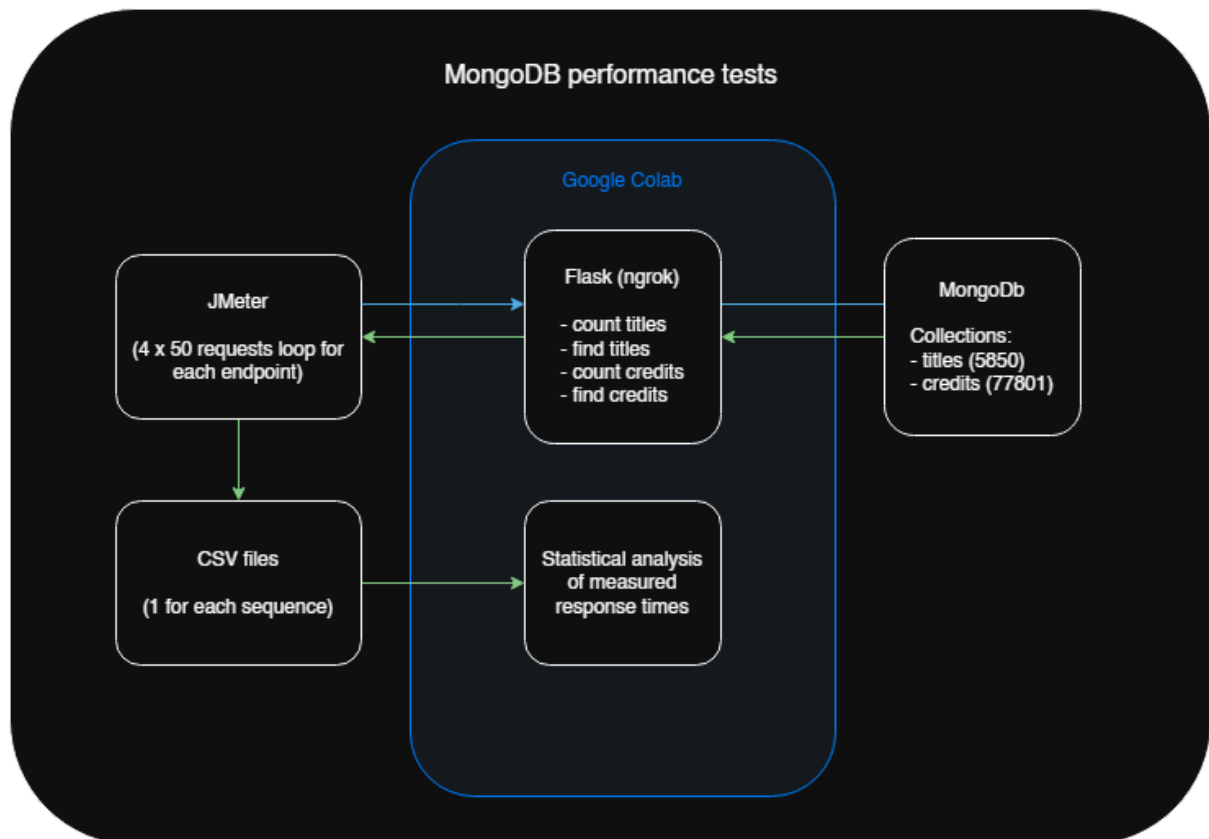
plt.figure()
plt.plot(decades_sorted, avg_cast, marker='o')

plt.xlabel("Dekada wydania")
plt.ylabel("Średnia liczba rekordów credits")
plt.title("Zmiana średniej liczby credits na tytuł w kolejnych dekadach")

plt.show()
```



4.2 Opis stanowiska do analiz



Stanowisko badawcze składało się z następujących elementów:

MongoDB Atlas - chmurowy klaster bazodanowy:

- o **`netflix-tv-shows-and-movies`** - baza 1
- o **`titles`** - kolekcja 1 (bazy 1) - 5850 dokumentów
- o **`credits`** - kolekcja 1 (bazy 1) - 77801 dokumentów

netflix-tv-shows-and-movies						
mongo-performance-cluster > netflix-tv-shows-and-movies						
View monitoring Visualize your data Create collection Refresh						
Collection name	Properties	Storage size	Documents	Avg. document size	Indexes	Total index size
credits	-	13.26 MB	78K	120.00 B	1	5.97 MB
titles	-	5.42 MB	5.9K	590.00 B	1	471.04 kB

Jupyter Notebook – środowisko analityczne do:

- przygotowania danych do testów (inspect, clean, insert),
- uruchomienia API (Flask) na publicznym adresie (ngrok),
- uruchamiania zapytań do bazy danych w chmurze,
- analizy statystycznej wyników.

Notatnik ``mongo-atlas-performance-test_netflix-db.ipynb`` realizuje kompletny proces przygotowania danych, konfiguracji środowiska chmurowego MongoDB Atlas oraz analizy wydajności zapytań do bazy danych. Poniżej opisano funkcjonalność poszczególnych sekcji notatnika.

*JMeter jest jedynym systemem zewnętrznym, który należy uruchomić lokalnie na maszynie i wykonać zapytania do API, które jest uruchamiane w notatniku i udostępniane pod publiczny adresem.

Zawartości notatnika:

1. Environment setup (konfiguracja środowiska)

- instalacja biblioteki pymongo
- połączenie z MongoDB Atlas przez MongoClient
- pobranie danych logowania z Google Colab Secrets (userdata.get(...))
- zdefiniowanie stałych: nazwa bazy, nazwy kolekcji, ścieżki i parametry datasetu

Odpowiedzialność:

- przygotowuje środowisko uruchomieniowe w Colab,
- tworzy i weryfikuje bezpieczne połączenie z MongoDB Atlas (ping),
- ustala wspólne stałe (np. DB_NAME, nazwy kolekcji titles, credits, parametry datasetu).

2. Create Database and Collections (utworzenie bazy i kolekcji + załadowanie danych)

- pobranie datasetu z Kaggle (instalacja kaggle, generowanie kaggle.json z Colab Secrets: KAGGLE_USER, KAGGLE_KEY)
- pobranie i rozpakowanie plików CSV (titles.csv, credits.csv)
- utworzenie bazy i kolekcji w MongoDB (titles, credits)
- przygotowanie danych w Pandas (inspekcja typów, podgląd)
- czyszczenie danych (m.in. konwersje, naprawa struktur; np. parsing pól listowych)
- wstawienie dokumentów do MongoDB (insert_many) + pomiar czasu inserta

Odpowiedzialność:

- zapewnia źródło danych do testów (pobranie i przygotowanie),
- buduje strukturę testową w MongoDB Atlas: baza + 2 kolekcje,
- gwarantuje, że dane są w formacie “Mongo-ready” (dokumenty), zanim trafią do chmury.

3. Data Visualisation (wizualizacje danych)

- pobieranie dokumentów z MongoDB Atlas (find z projekcją pól)
- wizualizacje opisowe (m.in. top 10 krajów produkcji, top 10 gatunków, udział filmów vs seriali)
- zależności typu: liczba tytułów vs średni runtime (dla krajów) – przygotowanie danych i wykres (bąbelkowy)
- analiza kolekcji credits: top aktorów, rozkład ról, oraz elementy łączenia informacji “titles + credits” na poziomie DataFrame
- join both collections (titles & credits)

Odpowiedzialność:

- pokazuje, że dane są poprawnie załadowane i da się je analizować (walidacja “biznesowa” zbioru),
- dostarcza gotowe wykresy do raportu (część “charakterystyka danych”),

- wykonuje analizę statystyczno-opisową niezależną od JMetera (czyli: “co mamy w bazie”).

4. Performance tests (warstwa testów wydajności + analiza wyników JMeter)

Ta sekcja ma dwie role:

4.1 - 4.3 Udostępnienie API do testów (ngrok + Flask)

- instalacja flask i pyngrok
- konfiguracja ngrok (NGROK_AUTHTOKEN) i wystawienie publicznego URL (tunel na port 5000)
- aplikacja Flask z endpointami:
 - / (healthcheck)
 - /titles i /credits – zapytania find() z parametrem limit z query string (domyślnie 10)
 - analogiczne operacje count_documents({}) dla kolekcji
- pomiar czasu odpowiedzi (w ms) po stronie serwera (time.perf_counter())

Odpowiedzialność:

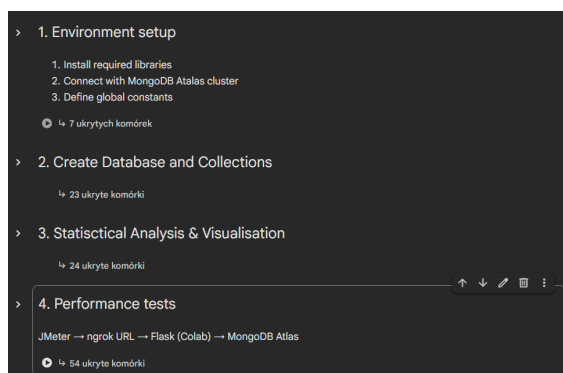
- tworzy “most” testowy: JMeter → publiczny ngrok URL → Flask → MongoDB Atlas,
- standaryzuje operacje (find/count) i mierzy czas odpowiedzi po stronie backendu,
- umożliwia testowanie dwóch kolekcji w identyczny sposób (porównywalność).

4.4 Analiza wyników testów JMeter

- przygotowanie katalogu na pliki JMetera (/content/jmeter)
- wczytanie 2 plików CSV wyeksportowanych z JMetera (elapsed times), przeliczenie ms na s
- pipeline analityczny dla testów:
 - histogramy (sprawdzenie rozkładu),
 - IQR (usuwanie wartości odstających),
 - Shapiro–Wilk (normalność),
 - Levene (równość wariancji),
 - t-test / Welch (w zależności od wyniku Levene),
 - podsumowanie wniosków (czy różnice są istotne statystycznie)

Odpowiedzialność:

- “zamyka pętlę projektu”: z surowych wyników JMetera robi dowód statystyczny istotności różnic,
- zapewnia metodyczne porównanie wydajności między kolekcjami/operacjami zgodnie z wymaganiem tematu.



API Flask – warstwa pośrednia umożliwiająca generowanie zapytań, wystawiająca 5 endpoint'ów HTTP:

- GET `/` - API healthcheck
- GET `/titles?limit=x` - find* <x> dokumentów z bazy titles (testy wykonano dla x = 1000)
- GET `/titles/count` - count dokumentów z bazy titles
- GET `/credits?limit=x` - find* <x> dokumentów z bazy credits (testy wykonano dla x = 1000)
- GET `/credits/count` - count dokumentów z bazy credits

*Adnotacja: endpoint uruchamiający `find` faktycznie uruchamia zapytanie do bazy, ale zwraca tylko ilość pobranych dokumentów, aby nie obciążać API co wydłużyłoby czas odpowiedzi HTTP i pogorszyłoby jakość wyników testów wydajności samej bazy.

```
4.3 Flask API

db = client[DB_NAME]

# Select collections
titles_col = db[TITLES]
credits_col = db[CREDITS]

from flask import Flask, request, jsonify
from threading import Thread
import random
import logging
import time

app = Flask(__name__)

@app.route("/", methods=["GET"])
def healthcheck():
    return "OK - MongoDB Performance Test API"

@app.route("/titles", methods=["GET"])
def titles():
    limit = request.args.get("limit", default=10, type=int)

    if limit <= 0:
        return "", 400

    start = time.perf_counter()
    documents = list(
        titles_col.find({}).limit(limit)
    )
    elapsed_ms = (time.perf_counter() - start) * 1000

    return jsonify({
        "collection": "titles",
        "operation": "find_documents",
        "documents": len(documents),
        "response_time_ms": round(elapsed_ms, 3)
    })

@app.route("/titles/count", methods=["GET"])
def titles_count():
    start = time.perf_counter()
    count = titles_col.count_documents({})
    elapsed_ms = (time.perf_counter() - start) * 1000

    return jsonify({
        "collection": "titles",
        "operation": "count_documents",
        "documents": count,
        "response_time_ms": round(elapsed_ms, 3)
    })

@app.route("/credits", methods=["GET"])
def credits():
    limit = request.args.get("limit", default=10, type=int)

    if limit <= 0:
        return "", 400

    start = time.perf_counter()
    documents = list(
        credits_col.find({}).limit(limit)
    )
    elapsed_ms = (time.perf_counter() - start) * 1000

    return jsonify({
        "collection": "credits",
        "operation": "find_documents",
        "documents": len(documents),
        "response_time_ms": round(elapsed_ms, 3)
    })

@app.route("/credits/count", methods=["GET"])
def credits_count():
    start = time.perf_counter()
    count = credits_col.count_documents({})
    elapsed_ms = (time.perf_counter() - start) * 1000

    return jsonify({
        "collection": "credits",
        "operation": "count_documents",
        "documents": count,
        "response_time_ms": round(elapsed_ms, 3)
    })

def run_flask():
    app.logger.setLevel(logging.INFO)
    app.run(host="0.0.0.0", port=5000, use_reloader=False, threaded=True)

run_flask()

... Pokaż ukryte dane wyjściowe
```

Konfiguracja ngrok:

```
4.2 ngrok

Configure ngrok authentication and open a public tunnel for local services on port 5000 .

[25] ✓ 5 s from pyngrok import ngrok

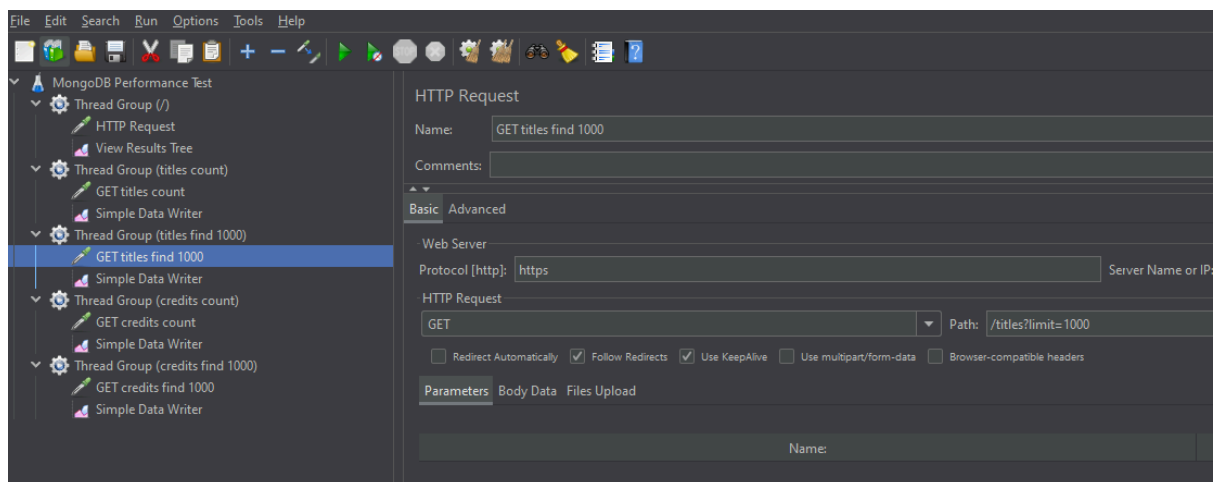
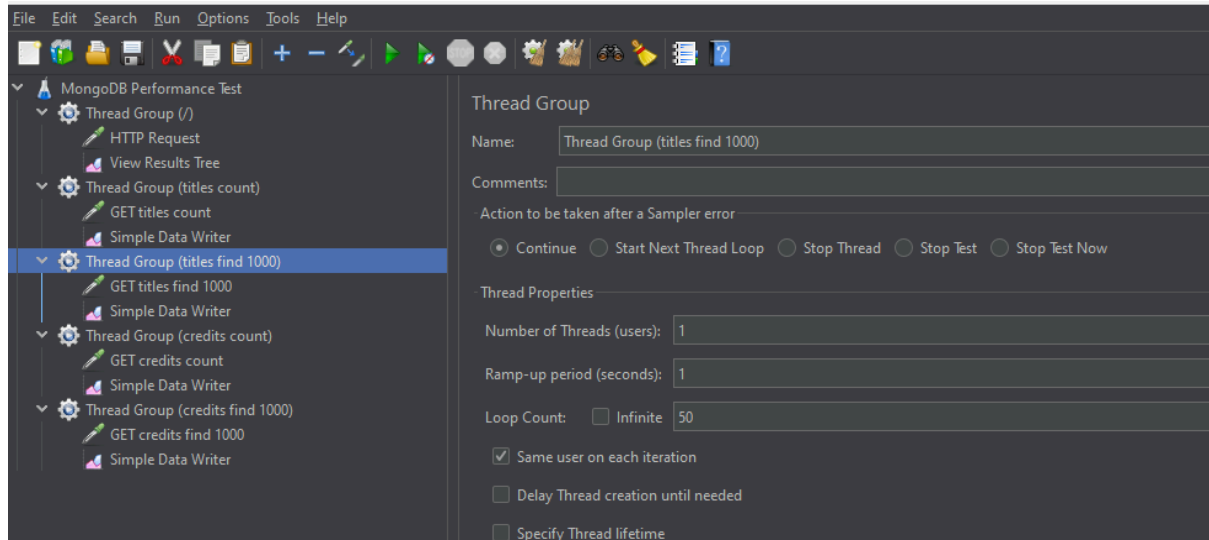
ngrok_authtoken = userdata.get('NGROK_AUTHTOKEN')
ngrok.set_auth_token(ngrok_authtoken)
del ngrok_authtoken

public_url = ngrok.connect(5000)
print(f"Public URL: {public_url}")
```

JMeter – Thread Groups rządań HTTP

Wykonuje 4 sekwencje, każda po 50 żądań HTTP na każdy z endpoint'ów Flask API i zapisuje wyniki każdej serii do oddzielnego pliku csv:

- `jmeter\_titles\_find\_1000\_results.csv` - GET `{{colab\_ngrok\_url}}/titles?limit=1000`
- `jmeter\_titles\_count\_results.csv` - GET `{{colab\_ngrok\_url}}/titles/count`
- `jmeter\_credits\_find\_1000\_results.csv` - GET `{{colab\_ngrok\_url}}/credits?limit=1000`
- `jmeter\_credits\_count\_results.csv` - GET `{{colab\_ngrok\_url}}/credits/count`



Name	Date modified	Type	Size
jmeter_credits_count_results.csv	13.01.2026 18:06	Microsoft Excel C...	3 KB
jmeter_credits_find_1000_results.csv	13.01.2026 21:37	Microsoft Excel C...	3 KB
jmeter_titles_count_results.csv	13.01.2026 18:06	Microsoft Excel C...	3 KB
jmeter_titles_find_1000_results.csv	13.01.2026 21:36	Microsoft Excel C...	3 KB
mongodb-performance-test.jmx	13.01.2026 23:55	JMX File	17 KB

4.3 Metodyka badań wydajności

Przeprowadzono 2 testy - 2 zapytania na test:

Test wydajności 1: `count`

- **Q1:** count - policzenie dokumentów z kolekcji **titles**,
- **Q2:** count - policzenie dokumentów z kolekcji **credits**.

Test wydajności 2: `find 1000`

- **Q1:** find 1000 - pobranie dokumentów z kolekcji **titles**,
- **Q2:** find 1000 - pobranie dokumentów z kolekcji **credits**.

Dla każdego zapytania:

- wykonano serię pomiarów czasów odpowiedzi (JMeter),
- obliczono **średnią i medianę**,
- sformułowano hipotezy statystyczne:

H0: Brak istotnej różnicy pomiędzy średnimi czasami odpowiedzi

H1: Istnieje istotna różnica pomiędzy średnimi czasami odpowiedzi

Test wydajności 1 - porównanie czasów zapytań count_documents:

Endpointy wykorzystane przez JMeter:

```
@app.route("/titles/count", methods=["GET"])
def titles_count():
    start = time.perf_counter()
    count = titles_col.count_documents({})
    elapsed_ms = (time.perf_counter() - start) * 1000

    return jsonify({
        "collection": "titles",
        "operation": "count_documents",
        "documents": count,
        "response_time_ms": round(elapsed_ms, 3)
    })

@app.route("/credits/count", methods=["GET"])
def credits_count():
    start = time.perf_counter()
    count = credits_col.count_documents({})
    elapsed_ms = (time.perf_counter() - start) * 1000

    return jsonify({
        "collection": "credits",
        "operation": "count_documents",
        "documents": count,
        "response_time_ms": round(elapsed_ms, 3)
    })
```

Przed testem: wgranie plików eksportowanych z JMeter do notebook'a

```
Test 1: Analiza zapytania count_documents
Dane (zmierzone czasy zapytań) pochodzą z plików:
• jmeter_titles_count_results.csv
• jmeter_credits_count_results.csv

JMMETER_TITLES_RESULTS_CSV = "jmeter_titles_find_1000_results.csv"
JMMETER_CREDITS_RESULTS_CSV = "jmeter_credits_find_1000_results.csv"

import pandas as pd

titles_jmeter_df = pd.read_csv(f"{JMMETER_DIR}/{JMMETER_TITLES_RESULTS_CSV}")
credits_jmeter_df = pd.read_csv(f"{JMMETER_DIR}/{JMMETER_CREDITS_RESULTS_CSV}")

print("Titles DF shape:", titles_jmeter_df.shape)
print("Credits DF shape:", credits_jmeter_df.shape)

print("\nColumns:", titles_jmeter_df.columns.tolist())

titles_times_ms = titles_jmeter_df["elapsed"].tolist()
credits_times_ms = credits_jmeter_df["elapsed"].tolist()

titles_times = [t / 1000 for t in titles_times_ms]
credits_times = [t / 1000 for t in credits_times_ms]

print("\nFirst 5 response times (titles) [s]:", titles_times[:5])
print("First 5 response times (credits) [s]:", credits_times[:5])

Titles DF shape: (50, 4)
Credits DF shape: (50, 4)

Columns: ['timestamp', 'elapsed', 'label', 'success']

First 5 response times (titles) [s]: [1.07, 0.587, 0.582, 0.596, 0.593]
First 5 response times (credits) [s]: [0.89, 0.582, 0.573, 0.576, 0.575]
```

Kroki testu wydajności 1 `count`:

- (Test 1 `count`) Krok 1: Weryfikacja rozkładu wyników - histogram
- (Test 1 `count`) Krok 2: IQR - usunięcie wartości odstających
- (Test 1 `count`) Krok 3: Test Shapiro-Wilk
- (Test 1 `count`) Krok 4: Test Levene
- (Test 1 `count`) Krok 5: Test t-Student

(Test 1 `count`) Krok 1: Weryfikacja rozkładu wyników - histogram

```
Step 1: Weryfikacja rozkładu wyników - histogram

import statistics as stats
import matplotlib.pyplot as plt

print("=== TITLES ===")
print("Mean [s]:", round(stats.mean(titles_times), 4))
print("Median [s]:", round(stats.median(titles_times), 4))
print("Std dev [s]:", round(stats.stdev(titles_times), 4))

print("\n=== CREDITS ===")
print("Mean [s]:", round(stats.mean(credits_times), 4))
print("Median [s]:", round(stats.median(credits_times), 4))
print("Std dev [s]:", round(stats.stdev(credits_times), 4))

# Histograms
plt.figure(figsize=(12, 4))

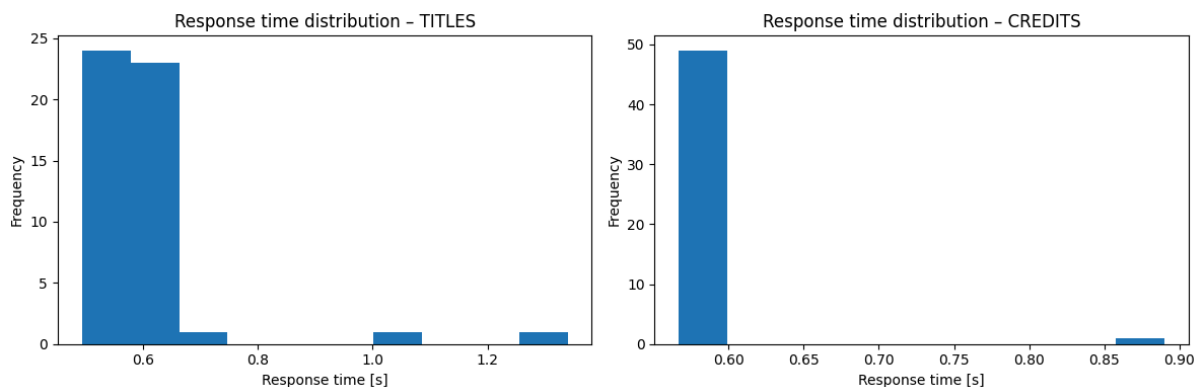
# Histogram - TITLES
plt.subplot(1, 2, 1)
plt.hist(titles_times, bins=10)
plt.title("Response time distribution - TITLES")
plt.xlabel("Response time [s]")
plt.ylabel("Frequency")

# Histogram - CREDITS
plt.subplot(1, 2, 2)
plt.hist(credits_times, bins=10)
plt.title("Response time distribution - CREDITS")
plt.xlabel("Response time [s]")
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()

*** === TITLES ===
Mean [s]: 0.5773
Median [s]: 0.581
Std dev [s]: 0.1411

=== CREDITS ===
Mean [s]: 0.5816
Median [s]: 0.574
Std dev [s]: 0.0448
```



Przykładowe powody outliers:

- cold start
- cache warm-up
- jitter chmurowy (Atlas free tier)

(Test 1 `count`) **Krok 2:** IQR - usunięcie wartości odstających

```
Step 2: IQR - usunięcie wartości odstających

import numpy as np

def remove_outliers_iqr(data):
    q1 = np.percentile(data, 25)
    q3 = np.percentile(data, 75)
    iqr = q3 - q1

    lower_bound = q1 - 1.5 * iqr
    upper_bound = q3 + 1.5 * iqr

    filtered = [x for x in data if lower_bound <= x <= upper_bound]
    return filtered, lower_bound, upper_bound

filtered_titles_times, lb_t, ub_t = remove_outliers_iqr(titles_times)
filtered_credits_times, lb_c, ub_c = remove_outliers_iqr(credits_times)

print("TITLES:")
print(f" Original size: {len(titles_times)}")
print(f" Filtered size: {len(filtered_titles_times)}")
print(f" Bounds: [{lb_t:.3f}, {ub_t:.3f}]")

print("\nCREDITS:")
print(f" Original size: {len(credits_times)}")
print(f" Filtered size: {len(filtered_credits_times)}")
print(f" Bounds: [{lb_c:.3f}, {ub_c:.3f}]")

TITLES:
Original size: 50
Filtered size: 48
Bounds: [0.366, 0.724]

CREDITS:
Original size: 50
Filtered size: 46
Bounds: [0.563, 0.587]
```

```
Histogramy 'Response time distribution' po filtracji IQR

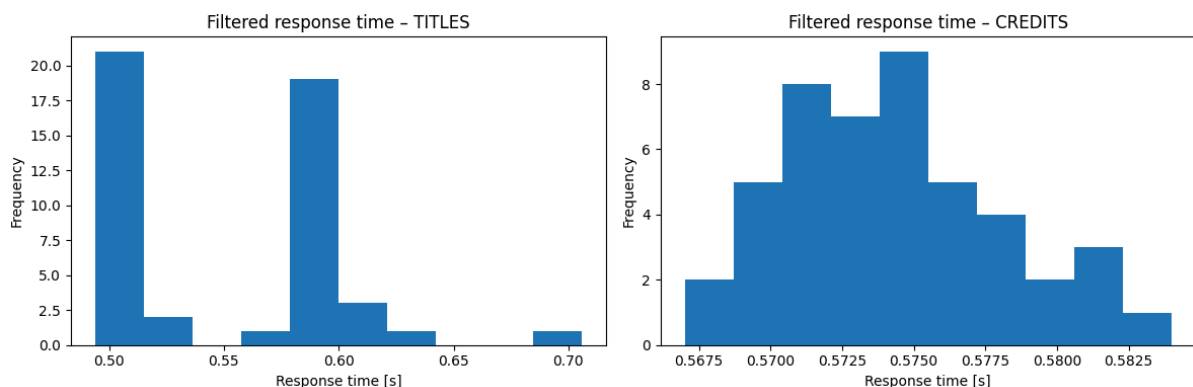
import matplotlib.pyplot as plt

plt.figure(figsize=(12, 4))

# Histogram - TITLES (after IQR)
plt.subplot(1, 2, 1)
plt.hist(filtered_titles_times, bins=10)
plt.title("Filtered response time - TITLES")
plt.xlabel("Response time [s]")
plt.ylabel("Frequency")

# Histogram - CREDITS (after IQR)
plt.subplot(1, 2, 2)
plt.hist(filtered_credits_times, bins=10)
plt.title("Filtered response time - CREDITS")
plt.xlabel("Response time [s]")
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()
```



(Test 1 `count`) Krok 3: Test Shapiro-Wilk

```
Step 3: Test Shapiro-Wilk

from scipy.stats import shapiro

shapiro_titles = shapiro(filtered_titles_times)
shapiro_credits = shapiro(filtered_credits_times)

print("Shapiro-Wilk TITLES p-value:", shapiro_titles.pvalue)
print("Shapiro-Wilk CREDITS p-value:", shapiro_credits.pvalue)

*** Shapiro-Wilk TITLES p-value: 4.099275338680995e-06
    Shapiro-Wilk CREDITS p-value: 0.22180492989312278
```

Test Shapiro-Wilka nie wykazał istotnych statystycznie odchyleń od rozkładu normalnego dla obu analizowanych zbiorów danych ($p > 0.05$), co uzasadnia zastosowanie parametrycznych testów istotności.

(Test 1 `count`) Krok 4: Test Levene

```
Step 4: Test Levene

from scipy.stats import levene

levene_test = levene(filtered_titles_times, filtered_credits_times)

print("Levene test p-value:", levene_test.pvalue)

Levene test p-value: 1.94226971325638e-12
```

Interpretacja testu Levene'a: **Levene p-value > 0.05**

- brak podstaw do odrzucenia hipotezy o równości wariancji
- wariancje obu grup można uznać za równe

Decyzja metodologiczna: Stosujemy klasyczny test **t-Student** dla prób niezależnych

(Test 1 `count`) Krok 5: Test t-Student

```
Step 5: Test t-Student

[20] ✓ 0 s
from scipy.stats import ttest_ind

t_test = ttest_ind(
    filtered_titles_times,
    filtered_credits_times,
    equal_var=True
)

print("t-statistic:", t_test.statistic)
print("t-test p-value:", t_test.pvalue)

print()

if (t_test.pvalue <= alpha):
    print(H1)
else:
    print(H0)

t-statistic: -3.097631539874763
t-test p-value: 0.0025868013576068882

H1: Istnieje istotna różnica pomiędzy średnimi czasami odpowiedzi
```

Test wydajności 2 - porównanie czasów zapytań `find.limit(1000)`:

Endpointy wykorzystane przez JMeter:

```
@app.route("/titles", methods=["GET"])
def titles():
    limit = request.args.get("limit", default=10, type=int)

    if limit <= 0:
        return "", 400

    start = time.perf_counter()
    documents = list(
        titles_col.find({}).limit(limit)
    )
    elapsed_ms = (time.perf_counter() - start) * 1000

    return jsonify({
        "collection": "titles",
        "operation": "find_documents",
        "documents": len(documents),
        "response_time_ms": round(elapsed_ms, 3)
    })
|

@app.route("/credits", methods=["GET"])
def credits():
    limit = request.args.get("limit", default=10, type=int)

    if limit <= 0:
        return "", 400

    start = time.perf_counter()
    documents = list(
        credits_col.find({}).limit(limit)
    )
    elapsed_ms = (time.perf_counter() - start) * 1000

    return jsonify({
        "collection": "credits",
        "operation": "find_documents",
        "documents": len(documents),
        "response_time_ms": round(elapsed_ms, 3)
    })
```

Przed testem: wgranie plików eksportowanych z JMeter do notebook'a

```
Test 2: Analiza zapytania find 1000 documents

Dane (zmierzone czasy zapytań) pochodzą z plików:
• jmeter_titles_find_1000_results.csv
• jmeter_credits_find_1000_results.csv

JMMETER_TITLES_RESULTS_CSV = "jmeter_titles_find_1000_results.csv"
JMMETER_CREDITS_RESULTS_CSV = "jmeter_credits_find_1000_results.csv"

import pandas as pd

titles_jmeter_df = pd.read_csv(f"{JMMETER_DIR}/{JMMETER_TITLES_RESULTS_CSV}")
credits_jmeter_df = pd.read_csv(f"{JMMETER_DIR}/{JMMETER_CREDITS_RESULTS_CSV}")

print("Titles DF shape:", titles_jmeter_df.shape)
print("Credits DF shape:", credits_jmeter_df.shape)

print("\nColumns:", titles_jmeter_df.columns.tolist())

titles_times_ms = titles_jmeter_df["elapsed"].tolist()
credits_times_ms = credits_jmeter_df["elapsed"].tolist()

titles_times = [t / 1000 for t in titles_times_ms]
credits_times = [t / 1000 for t in credits_times_ms]

print("\nFirst 5 response times (titles) [s]:", titles_times[:5])
print("First 5 response times (credits) [s]:", credits_times[:5])

...
Titles DF shape: (50, 4)
Credits DF shape: (50, 4)

Columns: ['timestamp', 'elapsed', 'label', 'success']

First 5 response times (titles) [s]: [1.07, 0.587, 0.582, 0.596, 0.593]
First 5 response times (credits) [s]: [0.89, 0.582, 0.573, 0.576, 0.575]
```

Kroki testu wydajności 2 `find 1000`:

- (Test 2 `find 1000`) **Krok 1:** Weryfikacja rozkładu wyników - histogram
- (Test 2 `find 1000`) **Krok 2:** IQR - usunięcie wartości odstających
- (Test 2 `find 1000`) **Krok 3:** Test Shapiro-Wilk
- (Test 2 `find 1000`) **Krok 4:** Test Levene
- (Test 2 `find 1000`) **Krok 5:** Test Welch

(Test 2 `find 1000`) **Krok 1:** Weryfikacja rozkładu wyników - histogram

Step 1: Weryfikacja rozkładu wyników - histogram

```
import statistics as stats
import matplotlib.pyplot as plt

print("=== TITLES ===")
print("Mean [s]:", round(stats.mean(titles_times), 4))
print("Median [s]:", round(stats.median(titles_times), 4))
print("Std dev [s]:", round(stats.stdev(titles_times), 4))

print("\n=== CREDITS ===")
print("Mean [s]:", round(stats.mean(credits_times), 4))
print("Median [s]:", round(stats.median(credits_times), 4))
print("Std dev [s]:", round(stats.stdev(credits_times), 4))

# Histograms
plt.figure(figsize=(12, 4))

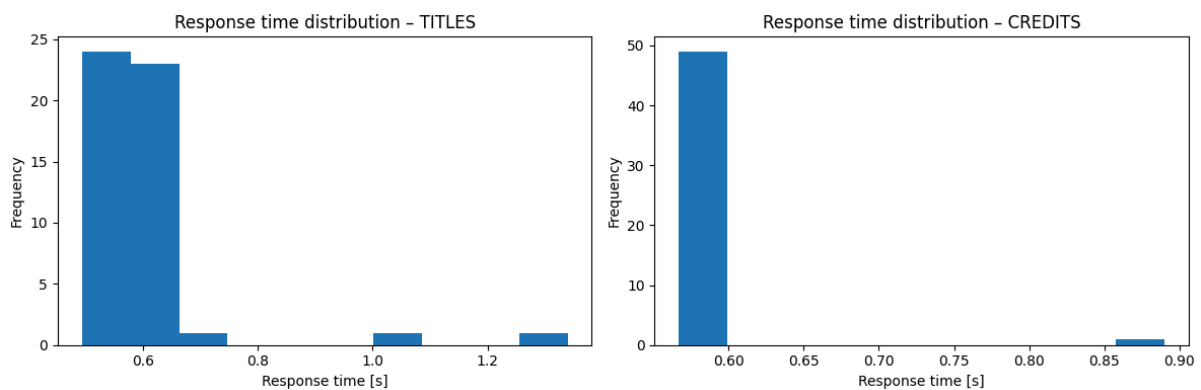
# Histogram - TITLES
plt.subplot(1, 2, 1)
plt.hist(titles_times, bins=10)
plt.title("Response time distribution - TITLES")
plt.xlabel("Response time [s]")
plt.ylabel("Frequency")

# Histogram - CREDITS
plt.subplot(1, 2, 2)
plt.hist(credits_times, bins=10)
plt.title("Response time distribution - CREDITS")
plt.xlabel("Response time [s]")
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()

*** === TITLES ===
Mean [s]: 0.5773
Median [s]: 0.581
Std dev [s]: 0.1411

=== CREDITS ===
Mean [s]: 0.5816
Median [s]: 0.574
Std dev [s]: 0.0448
```



Przykładowe powody outliers:

- cold start
- cache warm-up
- jitter chmurowy (Atlas free tier)

(Test 2 `find 1000`) **Krok 2:** IQR - usunięcie wartości odstających

Step 2: IQR - usunięcie wartości odstających

```
import numpy as np

def remove_outliers_iqr(data):
    q1 = np.percentile(data, 25)
    q3 = np.percentile(data, 75)
    iqr = q3 - q1

    lower_bound = q1 - 1.5 * iqr
    upper_bound = q3 + 1.5 * iqr

    filtered = [x for x in data if lower_bound <= x <= upper_bound]
    return filtered, lower_bound, upper_bound

filtered_titles_times, lb_t, ub_t = remove_outliers_iqr(titles_times)
filtered_credits_times, lb_c, ub_c = remove_outliers_iqr(credits_times)

print("TITLES:")
print(f" Original size: {len(titles_times)}")
print(f" Filtered size: {len(filtered_titles_times)}")
print(f" Bounds: [{lb_t:.3f}, {ub_t:.3f}]")

print("\nCREDITS:")
print(f" Original size: {len(credits_times)}")
print(f" Filtered size: {len(filtered_credits_times)}")
print(f" Bounds: [{lb_c:.3f}, {ub_c:.3f}]")

... TITLES:
    Original size: 50
    Filtered size: 48
    Bounds: [0.366, 0.724]

CREDITS:
    Original size: 50
    Filtered size: 46
    Bounds: [0.563, 0.587]
```

Histogramy 'Response time distribution' po filtracji IQR

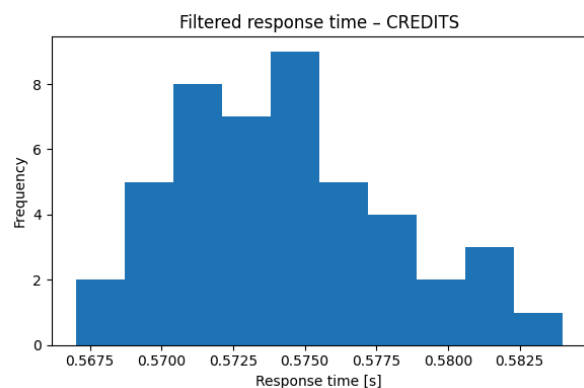
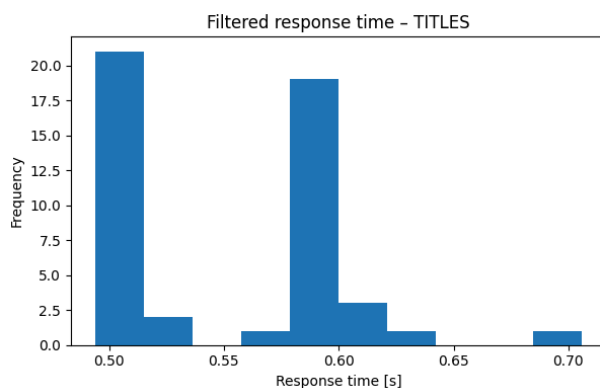
```
import matplotlib.pyplot as plt

plt.figure(figsize=(12, 4))

# Histogram - TITLES (after IQR)
plt.subplot(1, 2, 1)
plt.hist(filtered_titles_times, bins=10)
plt.title("Filtered response time - TITLES")
plt.xlabel("Response time [s]")
plt.ylabel("Frequency")

# Histogram - CREDITS (after IQR)
plt.subplot(1, 2, 2)
plt.hist(filtered_credits_times, bins=10)
plt.title("Filtered response time - CREDITS")
plt.xlabel("Response time [s]")
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()
```



(Test 2 `find 1000`) **Krok 3: Test Shapiro-Wilk**

Step 3: Test Shapiro-Wilk

```
from scipy.stats import shapiro

shapiro_titles = shapiro(filtered_titles_times)
shapiro_credits = shapiro(filtered_credits_times)

print("Shapiro-Wilk TITLES p-value:", shapiro_titles.pvalue)
print("Shapiro-Wilk CREDITS p-value:", shapiro_credits.pvalue)

Shapiro-Wilk TITLES p-value: 4.099275338680995e-06
Shapiro-Wilk CREDITS p-value: 0.22180492989312278
```

Test Shapiro-Wilka nie wykazał istotnych statystycznie odchyleń od rozkładu normalnego dla obu analizowanych zbiorów danych ($p > 0.05$), co uzasadnia zastosowanie parametrycznych testów istotności.

(Test 2 `find 1000`) **Krok 4: Test Levene**

Step 4: Test Levene

```
from scipy.stats import levene

levene_test = levene(filtered_titles_times, filtered_credits_times)

print("Levene test p-value:", levene_test.pvalue)

Levene test p-value: 1.94226971325638e-12
```

Interpretacja testu Levene'a: **Levene p-value < 0.05**

- odrzucenie hipotezy o równości wariancji
- wariancje są istotnie różne

Decyzja metodologiczna: stosujemy klasyczny test **Welch**

(Test 2 `find 1000`) **Krok 5: Test Welch**

Step 5 (if Levene p > 0.05): Test Welch

```
[28] ✓ 0 s
from scipy.stats import ttest_ind

welch_test = ttest_ind(
    filtered_titles_times,
    filtered_credits_times,
    equal_var=False
)

print("Welch t-statistic:", welch_test.statistic)
print("Welch p-value:", welch_test.pvalue)

print()

if (welch_test.pvalue <= alpha):
    print(H1)
else:
    print(H0)

Welch t-statistic: -3.1641767066027158
Welch p-value: 0.0027103334428839414

H1: Istnieje istotna różnica pomiędzy średnimi czasami odpowiedzi
```

4.4 Wyniki i wnioski

Na podstawie interpretacji 2 oddzielnych testów (Test 1 t-Studenta, Test 2 Welch):

- ODRZUCAMY hipotezę **H0**, POTWIERDZAMY alternatywną **H1**

ponieważ różnica średnich czasów odpowiedzi jest istotna statystycznie.

Obie serie (czasy zapytań **titles** i **credits**):

- miały rozkład normalny
- miały porównywalne wariancje

Różnice średnich:

- są systematyczne
- nie wynikają z losowego szumu
- są konsekwencją charakterystyki danych / kolekcji

Przeprowadzone testy dla prób niezależnych wykazał istotną statystycznie różnicę pomiędzy średnimi czasami odpowiedzi zapytań typu `count_documents({})`, `find.limit(1000)` wykonywanych na kolekcjach **titles** i **credits** (p-value < 0.05). Na tej podstawie odrzucono hipotezę zerową o równości średnich czasów odpowiedzi, potwierdzając wpływ charakterystyki zbioru danych na wydajność bazy MongoDB.

Wyniki:

- zapytania do kolekcji **credits** charakteryzowały się wyraźnie dłuższymi czasami odpowiedzi,
- większy wolumen dokumentów istotnie wpływał na wydajność,
- uzyskane wartości p-value pozwoliły na odrzucenie hipotezy H0 i potwierdzenie alternatywnej H1.

Wnioski:

- rozmiar kolekcji MongoDB ma istotny wpływ na czasy odpowiedzi,
- nawet w darmowej chmurze możliwe jest prowadzenie rzetelnych analiz wydajności,
- MongoDB dobrze nadaje się do analiz dokumentowych, jednak wymaga świadomego projektowania zapytań.

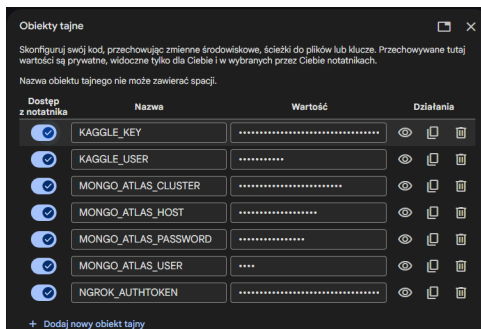
4.5 Instrukcja uruchamiania systemu

Notatnik został przygotowany tak, by w pełni automatyzować proces pozyskania, wgrania i testowania danych w celu łatwego odtworzenia systemu na innym środowisku.

Konfiguracja środowiska do pracy z notatnikiem wymaga zdefiniowania zmiennych środowiskowych jako Secrets, ponieważ kod w komórkach oczekuje ich i na podstawie ich wartości wykonuje połączenia.

(Pełne instrukcje zostały szczegółowo opisane w notatniku)

Zmienne, które należy skonfigurować w Google Colab:



Połączenie z MongoDB Atlas:

- `MONGO\_ATLAS\_USER`
- `MONGO\_ATLAS\_PASSWORD`
- `MONGO\_ATLAS\_CLUSTER`
- `MONGO\_ATLAS\_HOST`

```
from pymongo.mongo_client import MongoClient
from pymongo.server_api import ServerApi
from google.colab import userdata

user = userdata.get('MONGO_ATLAS_USER')
password = userdata.get('MONGO_ATLAS_PASSWORD')
cluster = userdata.get('MONGO_ATLAS_CLUSTER')
host = userdata.get('MONGO_ATLAS_HOST')

uri = f"mongodb+srv://{user}:{password}@{cluster}.{host}?retryWrites=true&w-majority&appName={cluster}"
# Create a new client and connect to the server
client = MongoClient(uri, server_api=ServerApi('1'))

# Send a ping to confirm a successful connection
try:
    client.admin.command('ping')
    print("Pinged your deployment. You successfully connected to MongoDB!")
except Exception as e:
    print(e)

del user, password, cluster, host, uri

... Pinged your deployment. You successfully connected to MongoDB!
```

Połączenie z kaggle:

Należy wcześniej utworzyć darmowe konto i wygenerować API token

- `KAGGLE\_USER`
- `KAGGLE\_KEY`

```
# Generate kaggle.json file
from google.colab import userdata
import json
import os

KAGGLE_DIR = "/root/.kaggle"
KAGGLE_API_TOKEN_FILE = f"{KAGGLE_DIR}/kaggle.json"

kaggle_creds = {
    "username": userdata.get('KAGGLE_USER'),
    "key": userdata.get('KAGGLE_KEY')
}

os.makedirs(KAGGLE_DIR, exist_ok=True)
with open(KAGGLE_API_TOKEN_FILE, "w") as f:
    json.dump(kaggle_creds, f)

os.chmod(KAGGLE_API_TOKEN_FILE, 0o600)

del kaggle_creds

print(f"✓ File '{KAGGLE_API_TOKEN_FILE}' created successfully!")

... ✓ File '/root/.kaggle/kaggle.json' created successfully!
```

```
Download and unzip dataset files

# Change directory to /content
%cd /content

# Create "data" if doesn't exist
import os
if not os.path.exists("data"):
    os.mkdir("data")
    print(f"Directory '{DATA_DIR}' created successfully.")
else:
    print(f"Directory '{DATA_DIR}' already exists.")

# Change directory to /data
%cd $DATA_DIR

# Download the dataset from Kaggle (using kaggle-cli)
!kaggle datasets download -d $DATASET

# Unzip downloaded dataset
!unzip -o $ZIP_NAME

# List downloaded dataset files
ls

... /content
  Directory '/content/data' created successfully.
  content/data
  Dataset URL: https://www.kaggle.com/datasets/victorspeiro/netflix-tv-shows-and-movies
  license(s): CC0 1.0
  Downloading netflix-tv-shows-and-movies.zip to /content/data
  0% 0.00/2.25M [00:00<, 76/s]
  100% 2.25M/2.25M [00:00<0m, 759M/s]
  Archive: netflix-tv-shows-and-movies.zip
  inflating: credits.csv
  inflating: titles.csv
  credits.csv netflix-tv-shows-and-movies.zip titles.csv
```

Połączenie z kaggle:

Należy wcześniej utworzyć darmowe konto i wygenerować auth token

- `'NGROK_AUTHTOKEN'`

4.2 ngrok

Configure `ngrok` authentication and open a public tunnel for local services on port `5000` .

```
from pyngrok import ngrok

ngrok_authtoken = userdata.get('NGROK_AUTHTOKEN')
ngrok.set_auth_token(ngrok_authtoken)
del ngrok_authtoken

public_url = ngrok.connect(5000)
print(f"Public URL: {public_url}")
```

*Plik ze skonfigurowanym scenariuszem testów JMeter w załączniku.

5. Spis literatury

MongoDB Documentation - <https://www.mongodb.com/docs>

MongoDB Atlas - <https://www.mongodb.com/atlas>

Kaggle Datasets - <https://www.kaggle.com>

Netflix Movies and TV Shows - <https://www.kaggle.com/datasets/victorsoeiro/netflix-tv-shows-and-movies>

Student, W. (1908). The probable error of a mean. Biometrika.

Python Documentation – <https://docs.python.org>

Datablist - customers, organizations - <https://www.datablist.com/learn/csv/download-sample-csv-files>

Apache JMeter Documentation – <https://jmeter.apache.org/documentation.html>

6. Spis załączników

- Obsługa dużych zbiorów danych.pdf - dokumentacja projektowa (PDF)
- ``mongo-atlas-performance-test_netflix-db.ipynb`` - notatnik Jupyter z analizą
- ``titles.csv`` - zbiór danych
- ``credits.csv`` - zbiór danych
- ``mongodb-performance-test.jmx`` - plik jmeter z konfiguracją planów testów
- ``jmeter_netflix-db`` - folder zawierający pliki .csv z wynikami testów